

Lab 01

Introduction

This assignment will introduce you to the software and hardware development tools you will use during the semester to work with the STM32F429I-Discovery development board and the Cortex-M4 ARM Processor so that you may run programs using Keil Studio Cloud. The example code provided with this assignment shows you:

1. How to setup a project using Keil Studio
2. How to use timer interrupts to interface with the LEDs in the STM32F429-Discovery board using the “user push-button” on the board.

Prelab

Reading: This lab combines elements from Chapters 1-6 of the course textbook. You should read and understand those chapters. Note that we will focus on C++ in this course, so you do not need to digest the sections on C or MicroPython. Pay specific attention to Chapter 5 (Interrupts and Power Management) and Chapter 6 (Timing Operations).

Hardware

- ▶ STM32F429-Discovery
- ▶ Mini-USB

The STM32F429-Discovery is powered via the mini-USB connection and no external power supply will be needed although this option is available. When using the mini-USB connection to power the board you must be very careful that any other peripherals that are using either the 3V or 5V output do not exceed 100mA otherwise you will burn the mini-USB interface on the board.

Setup Procedure

1. Log in to Keil Studio Cloud
2. Connect your STM32F429-Discovery board to a USB port of your choice. Let Windows try to find the drivers. If the drivers are not found then the st-link usb drivers are missing and you will need assistance from the TAs.
3. Create new project called "MT2TA4-2024-Lab-01". Go to File -> New -> Mbed Project and select "empty Mbed OS project" from *Example projects* the dropdown. Name the project and uncheck "Initialize this project as a Git repository".
4. Set *Active project* to MT2TA4-2024-Lab-01
5. Set *Build target* to DISCO-F429ZI
6. Set *Connected device* to DISCO-F429ZI
7. Copy and paste the code from this lab's main.cpp file into main.cpp within your Keil Studio project.
8. Compile and Run the program using the run button:

This will compile and download the project to the discovery board. You should see LED4 (red) blink once per second. We call this *phase 0* in the code. Pressing the user (blue) button will make

Lab Requirements

Modify the demo code downloaded above to achieve the exact same behaviour using timer interrupts instead of software waits. The while loop in your main method should now be empty.

Marking Scheme

- ▶ **40 pts** A functional program that implements all the requirements correctly.
- ▶ **10 pts** Motivate your design and implementation decisions to your TA and answer questions about your code.
- ▶ **3 pts** main.cpp source file uploaded to your Avenue drop box for lab-01. In the event your project isn't fully functional, this may be used to justify partial marks.

Be prepared to demo the program you wrote to your TA in lab and also to defend your design and implementation decisions.

Useful Keil Studio APIs

DigitalOut | InterruptIn | Ticker | Timeout |

Hint

See textbook code Listing 6.6 on page 145 for an example of how to use timer interrupts.